Федеральное государственное автономное образовательное учреждение высшего образования «Национальный исследовательский университет ИТМО»

Факультет Программной Инженерии и Компьютерной Техники

Лабораторная работа №1

«Решение системы линейных алгебраических уравнений СЛАУ»

Вариант 8

Метод Гаусса с выбором главного элемента по столбцам

Выполнил:

Козаченко Данил Александрович

Группа P3212

Проверил:

Малышева Татьяна Алексеевна

Санкт-Петербург 2025

Содержание

Цель работы **3**

Описание метода, расчетные формулы **3**

Листинг программы **3**

Примеры и результаты работы программы **6**

Выводы **6**

Цель работы

Изучить численные методы решения СЛАУ, реализовать один из них на любом языке программирования.

Описание метода, расчетные формулы

Метод Гаусса с выбором главного элемента является модификацией стандартного метода Гаусса, позволяющей избегать вычислительной погрешности при значениях ведущих элементов, близких к нулю по абсолютной величине. Это реализуется благодаря выбору такой перестановки уравнений, что на k-ом шаге исключения ведущим элементом a_{ii} оказывался наибольший по модулю элемент k-го столбца.

То есть на шаге k выбирают «главный элемент по столбцу» в уравнениях с k до n выбирают в столбце k максимальный по модулю элемент и меняют строки i и k местами. Аналогично выбирается главный элемент по строке

Листинг программы

```
import numpy as np


def gauss_method(n, matrix):
    swaps = 0
    for i in range(n):
        max_in_col_idx = i
        for j in range(i + 1, n):
            if abs(matrix[j][i]) > abs(matrix[max_in_col_idx][i]):
                max_in_col_idx = j
        if max_in_col_idx != i:
            matrix[i], matrix[max_in_col_idx] = matrix[max_in_col_idx],
matrix[i]
            swaps += 1
        for j in range(i + 1, n):
            factor = matrix[j][i] / matrix[i][i]
            for k in range(i, n + 1):
                matrix[j][k] -= factor * matrix[i][k]
    print("Преобразованная треугольная матрица вместе со столбцом B:")
    for i in range(n):
        print([round(x, 4) for x in matrix[i][:-1]], "|", round(matrix[i][-
1], 4))

    det = (-1) ** swaps
    for i in range(n):
        det *= matrix[i][i]
    print(f"\nОпределитель: {det}")
    if det == 0:
        print("Система не имеет единственного решения")
        return None, 0

    x = [0] * n
    for i in range(n - 1, -1, -1):
        sum_ax = sum(matrix[i][j] * x[j] for j in range(i + 1, n))
        x[i] = (matrix[i][n] - sum_ax) / matrix[i][i]

    print("Вектор неизвестных (x):")
    for i in range(n):
        print(f"x{i + 1} = {round(x[i], 4)}")
```

```
    return x, det


def nevyazka(n, matrix, x):
    print("Вектор невязок:")
    r = [0] * n
    for i in range(n):
        ax = sum(matrix[i][j] * x[j] for j in range(n))
        r[i] = ax - matrix[i][n]
    print([float(r[i]) for i in range(n)])


def check(n, matrix):
    print("\n-----------Проверим результат с помощью NumPy-----------")
    A = np.array([row[:-1] for row in matrix])
    B = np.array([row[-1] for row in matrix])
    try:
        x_np = np.linalg.solve(A, B)
        det_np = np.linalg.det(A)
        print(f"Определитель: {round(det_np, 4)}")
        print("Вектор неизвестных (x):")
        for i in range(n):
            print(f"x{i + 1} = {round(x_np[i], 4)}")
        nevyazka(n, matrix, x_np)
    except np.linalg.LinAlgError:
        print("Система не имеет единственного решения (NumPy)")


def file_input():
    print("""!!! Убедитесь, что содержимое файла соответствует виду:
          n
          a_{11} a_{12} ... a_{1n} b_{1}
          a_{21} a_{22} ... a_{2n} b_{2}
          ...
          a_{n1} a_{n2} ... a_{nn} b_{n}""")
    f = input("Введите название файла: ")
    try:
        with open(f) as file:
            n = int(file.readline())
            if n > 20:
                print("Размерность матрицы должна быть не больше 20")
                return None, None
            matrix = []
            for i in range(n):
                row = list(map(float, file.readline().split()))
                matrix.append(row)
    except FileNotFoundError:
        print("Файл не найден")
        return None, None
    except ValueError:
        print("Неверный формат данных в файле")
        return None, None
    except Exception as e:
        print(f"Произошла ошибка: {e}")
        return None, None
    return n, matrix


def console_input():
    while True:
        n = input("Введите размерность матрицы (n<=20): ")
        if n.isdigit() and 1 <= int(n) <= 20:
            n = int(n)
```

```
            break
        else:
            print("Неверный ввод. Пожалуйста, введите целое число от 1 до
20")
    matrix = []
    print("Введите коэффициенты матрицы и столбца B (по формату: a11 a12 ...
a1n b1):")
    for i in range(n):
        while True:
            row_input = input(f"Строка {i + 1}: ")
            row = row_input.split()
            if len(row) == n + 1 and all(item.replace('.', '', 1).replace('-
', '', 1).isdigit() for item in row):
                matrix.append(list(map(float, row)))
                break
            else:
                print(f"Неверный ввод. Пожалуйста, введите {n + 1} чисел для
строки {i + 1}")
    return n, matrix


def main():
    while True:
        input_type = input("Выберите способ ввода данных: 1 - с клавиатуры, 2
- из файла: ")
        if input_type == '1':
            n, matrix = console_input()
            break
        elif input_type == '2':
            n, matrix = file_input()
            break
        else:
            print("Неверный выбор. Пожалуйста, выберите способ ввода данных")

    if matrix is not None:
        x, det = gauss_method(n, matrix)
        nevyazka(n, matrix, x)
        check(n, matrix)
    else:
        print("Ошибка при вводе данных. Программа завершена")


if __name__ == '__main__':
    main()
```

Примеры и результаты работы программы

```
Выберите способ ввода данных: 1 - с клавиатуры, 2 - из файла: 1
Введите размерность матрицы (n<=20): 3
Введите коэффициенты матрицы и столбца B (по формату: a11 a12 ... a1n b1):
Строка 1: 2 2 4 7
Строка 2: 6 9 3 5
Строка 3: 3 8 2 1
Преобразованная треугольная матрица вместе со столбцом B:
[6.0, 9.0, 3.0] | 5.0
[0.0, 3.5, 0.5] | -1.5
[0.0, 0.0, 3.1429] | 4.9048

Определитель: 66.0
Вектор неизвестных (x):
x1 = 1.0303
x2 = -0.6515
x3 = 1.5606
Вектор невязок:
[-8.881784197001252e-16, 0.0, 0.0]

-----------Проверим результат с помощью NumPy-----------
Определитель: 66.0
Вектор неизвестных (x):
x1 = 1.0303
x2 = -0.6515
x3 = 1.5606
Вектор невязок:
[-8.881784197001252e-16, 0.0, 0.0]
```

Можно заметить, что результат вычисления с помощью нашей программы и с помощью сторонней библиотеки полностью совпадает. Это связано с тем, что метод вычисления является точным и погрешность вычисления возникает только из-за проблемы хранения действительных чисел в памяти компьютера.

Выводы

Мы изучили различные методы решения СЛАУ и смогли реализовать один из них с помощью программы на Python.